The lux Tab-Bar Selection Discipline: a Z Specification

Formal model of the fire / echo / honour state machine

July 2026

1 Introduction

A `tab_bar` carries one Hub-authoritative selection — the *active* tab — and is rendered every frame by an ImGui tab bar that keeps its *own* idea of which tab is selected. The renderer must reconcile the two: it must *honour* the Hub's choice (force-select the declared active tab so ImGui's default of tab 0 cannot clobber it), and it must *fire* a `TabChanged` event when, and only when, a genuine user click moves ImGui's selection away from the active tab. An echo — the Hub's own value coming back round on the next frame — must fire nothing, or a `fire` → `Hub` → `re-push` → `fire` loop would run.

The renderer arbitrates with a small piece of per-render-session bookkeeping held in `WidgetState`: the *honoured* value, the active tab the frame last force-selected. Before anything is honoured the slot is empty (`_UNHONOURED`); `end()` writes the current active tab into it each frame; a whole-scene re-push resets it (`reset_honoured`), and a removal clears it (`discard_for`).

The honoured value alone is not enough. It records which *Hub* value was honoured; it does not record which *user* selection has already been reported. In the window between a user's click and the Hub's re-push — the click-to-re-push latency window — ImGui keeps reporting the clicked tab as selected while the active tab has not yet moved. With the honoured value equal to the active tab, every frame in that window reads as a fresh user switch and re-fires. Closing that defect is what forces a *second* slot, the *pending* selection: the tab for which a `TabChanged` is already outstanding. This document derives that slot and proves the four safety properties the design must have.

There is a further dimension the honoured write depends on: whether the bar *rendered* at all. A tab bar nested in a collapsed `collapsing_header`, or in a hidden window, has `begin_tab_bar` return *not opened*; its tab items never run, so no tab is force-selected and nothing can fire. The honoured write in `end()` must therefore take effect only on a frame that opened. Recording the active tab as honoured on a frame where nothing rendered is a stale honour of the same kind as an honour surviving a re-push: the frame the bar first becomes visible then reads the Hub value as already honoured, skips the first-frame force-select, and fires a spurious `TabChanged`. This is the fire/honour defect recurring for a fourth reason — not a lifecycle reset missed, but a honour recorded without a render — and the earlier model, which had no notion of a frame that does not render, could not exhibit it. The remedy is to gate the honoured write on the opened flag; this document extends the machine with that dimension so the gate can be proved to close the defect.

This is a finite state machine over one tab bar and a small fixed set of tab ids, so ProB can enumerate every interleaving of render frames, user clicks, re-pushes, and removals, and either confirm the invariants or produce the exact offending trace.

2 Basic Types

$[TABID]$

`TABID` is the set of tab identities the bar may hold. Two ids suffice to exhibit every defect: a declared active tab distinct from ImGui's tab-0 default, and a click that moves the selection between them. Three exercise the click-away-and-back paths more thoroughly. The carrier is bounded by ProB's `DEFAULT_SETSIZE`, so no explicit cardinality constant is needed.

3 Free Types and Constants

$ZBOOL ::= ztrue \mid zfalse$

A two-valued flag written as a free type rather than the toolkit `bool`, so ProB animates it as a small enumerated set.

$$MAXFIRE == 2$$

The fire counter saturates at `MAXFIRE`. A single click may fire at most once; a count of two is already a witness to the repeated-fire defect, so counting no higher keeps the state space small without hiding the violation.

$$home : TABID$$

`home` is ImGui's default selection, tab 0. A freshly rendered tab bar starts with ImGui showing `home`; the declared active tab may be some *other* id, and the first-frame honour must force the declared tab over this default.

4 State

The state is flat: one schema, seven components. Its predicate carries only *structural* invariants — that the two bookkeeping slots hold at most one id and the fire counter stays in range. These hold in every design, correct or buggy, so they act as coherence constraints, not as the safety properties under test. The safety properties (Section 9) are deliberately *absent* here, so that a violating state remains reachable and ProB can find it.

TabBar

$$\begin{array}{l} active : TABID \\ selected : TABID \\ honoured : \mathbb{P}\ TABID \\ pending : \mathbb{P}\ TABID \\ clicked : ZBOOL \\ fires : 0 \mathinner{..} MAXFIRE \\ spurious : ZBOOL \\ \hline \#honoured \leq 1 \\ \#pending \leq 1 \end{array}$$

`active` is the Hub-authoritative selection. `selected` is the tab ImGui currently reports open. `honoured` is the echo-suppression slot: empty for `_UNHONOURED`, or the singleton of the active tab the last frame force-selected. `pending` is the fire-suppression slot: empty when no fire is outstanding, or the singleton of the tab a `TabChanged` has already been fired for and not yet ratified by a re-push. Modelling the two slots as sets of size at most one lets the empty set stand for "none" without a sentinel value.

The remaining three components are pure bookkeeping for the invariant checks, not part of the renderer's own state. `clicked` records whether the current divergence between `selected` and `active` was produced by a genuine user gesture; a fire raised while `clicked = zfalse` is spurious. `fires` counts fires since the last click or re-push — the window in which at most one fire is legitimate. `spurious` latches `ztrue` the moment any spurious fire occurs.

5 Initialization

Init

$$\begin{array}{l} TabBar' \\ \hline active' = home \\ selected' = home \\ honoured' = \varnothing \\ pending' = \varnothing \\ clicked' = zfalse \\ fires' = 0 \\ spurious' = zfalse \end{array}$$

A single initial state: nothing honoured, nothing pending, ImGui and the Hub both on `home`, no fire seen.

6 Hub-Driven Operations

Three operations set the active tab from outside the renderer: the client declaring the bar, a whole-scene re-push of a surviving bar, and a removal followed by re-adding a same-id bar. All three reset the echo- and fire-suppression slots, because a fresh Hub value must be re-honoured from scratch. This reset is the guard that Section 11 removes to reproduce three of the four defects.

`Declare` models the client installing the bar with a declared active tab. ImGui starts a fresh widget, so `selected` is its tab-0 default, `home`; the declared active tab may differ.

Declare

$\Delta TabBar$
$v? : TABID$

$active' = v?$
$selected' = home$
$honoured' = \varnothing$
$pending' = \varnothing$
$clicked' = zfalse$
$fires' = 0$
$spurious' = spurious$

`Repush` models a whole-scene re-push of a bar that *survives* the re-push (`SceneReplica._replace_scene_state` calling `reset_honoured`). The ImGui widget persists, so its `selected` is kept; only the Hub value and the suppression slots change.

Repush

$\Delta TabBar$
$v? : TABID$

$active' = v?$
$selected' = selected$
$honoured' = \varnothing$
$pending' = \varnothing$
$clicked' = zfalse$
$fires' = 0$
$spurious' = spurious$

`RemoveReadd` models the bar being removed and a same-id bar re-added (`discard_for` clearing the slots). Its state effect is identical to `Repush` — a stale ImGui selection kept, the slots cleared — but it travels a different code path (`discard_for`, not `reset_honoured`). The two operations are kept distinct because each guards invariant 1 on a path the other does not cover; removing either guard alone must still be caught.

RemoveReadd

$\Delta TabBar$
$v? : TABID$

$active' = v?$
$selected' = selected$
$honoured' = \varnothing$
$pending' = \varnothing$
$clicked' = zfalse$
$fires' = 0$
$spurious' = spurious$

7 The User Click

A user clicks a tab other than the active one. ImGui moves its selection; the click opens the latency window (`clicked` becomes `ztrue`) and starts a fresh fire count. The click itself fires nothing — firing is the render frame's decision.

UserClick

$\Delta TabBar$
$u? : TABID$

$u? \neq active$
$selected' = u?$
$clicked' = ztrue$
$fires' = 0$
$active' = active$
$honoured' = honoured$
$pending' = pending$
$spurious' = spurious$

8 The Render Frame

Each frame the bar either *opens* — `begin_tab_bar` returns opened, the tab items run — or it does not. An opened frame is one of three cases; a not-opened frame is `FrameNotOpened`. One of the four is *always* enabled, so a render step never stalls: that is the reason the machine cannot deadlock (Section 9, invariant 4).

An opened frame runs `begin_tab` over every tab and then `end()`, and every opened case ends by writing the active tab into `honoured`: that is `end()`'s honoured write under its opened gate, and it is what makes the *next* frame read the current active tab as "already honoured". A not-opened frame runs no tab items and no force-select, and — this is the gate that closes the defect — its `end()` does *not* write `honoured`. The honoured slot is therefore only ever set by a frame that actually rendered and force-selected, so a non-empty `honoured` is a true witness that an opened frame has run.

`FrameForceSelect` is the fresh-value case: the active tab is not the honoured one, so `begin_tab` force-selects it. ImGui's selection is driven to the active tab, and no fire can occur — the only tab reported selected is the active tab itself, which is never a switch. This is both the first-frame honour and the echo suppression after a Hub-driven change.

FrameForceSelect

$\Delta TabBar$

$active \notin honoured$
$selected' = active$
$honoured' = \{active\}$
$pending' = pending$
$clicked' = zfalse$
$fires' = fires$
$spurious' = spurious$
$active' = active$

`FrameFire` is the genuine-switch case: the active tab is already honoured (no force-select this frame), ImGui reports a *different* tab selected, and no fire is yet outstanding for that tab. The frame fires, records the tab in `pending`, and bumps the counter. A fire raised while `clicked = zfalse` — no user gesture behind it — latches `spurious`.

FrameFire

$\Delta TabBar$

$active \in honoured$
$selected \neq active$
$selected \notin pending$
$selected' = selected$
$honoured' = \{active\}$
$pending' = \{selected\}$
$clicked' = clicked$
$fires' = \textbf{if } fires < MAXFIRE \textbf{ then } fires + 1 \textbf{ else } MAXFIRE$
$spurious' = \textbf{if } clicked = zfalse \textbf{ then } ztrue \textbf{ else } spurious$
$active' = active$

The guard `selected` $\notin$ `pending` is the fix. Once a tab is pending, later frames in the same window see it in `pending` and fall to `FrameSteady` instead of re-firing. Deleting that conjunct (and the `pending' = {selected}` write that feeds it) is the buggy variant of Section 11, defect (d).

`FrameSteady` is every other honoured frame: ImGui is on the active tab, or on a tab whose fire is already outstanding. Nothing fires; the honoured write still happens.

FrameSteady

$\Delta TabBar$

$active \in honoured$
$(selected = active \lor selected \in pending)$
$selected' = selected$
$honoured' = \{active\}$
$pending' = pending$
$clicked' = clicked$
$fires' = fires$
$spurious' = spurious$
$active' = active$

The three opened-frame guards partition the opened case: `active` $\notin$ `honoured` selects `FrameForceSelect`; `active` $\in$ `honoured` splits on the fire condition into `FrameFire` and `FrameSteady`. Exactly one of the three is enabled in every state.

`FrameNotOpened` is the frame that does not render: the bar is inside a collapsed `collapsing_header` or a hidden window, `begin_tab_bar` returns not opened, and no tab item runs. In the corrected design this leaves every component untouched — in particular `honoured`, because the honoured write is gated on the opened flag and does not fire this frame. It is enabled in *every* state, unguarded, modelling the fact that any frame may find the container collapsed.

FrameNotOpened

$\Delta TabBar$

$active' = active$
$selected' = selected$
$honoured' = honoured$
$pending' = pending$
$clicked' = clicked$
$fires' = fires$
$spurious' = spurious$

Because `FrameNotOpened` changes nothing, it adds no reachable state; its role is to make the not-opened frame an explicit, always-available step, so that the fidelity variant of Section 11 — which lets it write `honoured` — has somewhere to introduce the stale honour. With one of the three opened frames *and* `FrameNotOpened` enabled in every state, a render step is always available, so the opened dimension does not weaken deadlock freedom.

9 Invariants

Four safety properties must hold across all reachable states, plus deadlock-freedom. They are stated here as predicates over `TabBar`; how each is discharged is given in Section 10. None is placed in the `TabBar` schema: a predicate placed there would be enforced as a guard on every successor, silently disabling any operation that would break it and thereby masking the very defect this model exists to catch.

Invariant 1 — no spurious fire. A `TabChanged` is fired only on a genuine user switch, never off a Hub echo or a stale honoured value. Equivalently, `spurious` is never set:

$$spurious = zfalse.$$

This subsumes echo suppression — a Hub-driven change (`Declare`, `Repush`, `RemoveReadd`) carries `clicked = zfalse`, so any fire it provokes is spurious and would trip this invariant.

Invariant 2 — no repeated fire. A single user click yields at most one fire across all frames until the re-push. With the counter reset on every click and re-push:

$$fires \leq 1.$$

Invariant 3 — first-frame honour. Once an *opened* frame has rendered (`honoured` is non-empty), if the user has not clicked then ImGui shows the Hub's active tab — the declared active tab is never left clobbered by ImGui's tab-0 default:

$$honoured \neq \varnothing \land clicked = zfalse \Rightarrow selected = active.$$

The invariant reads `honoured` $\neq \varnothing$ as "an opened frame has force-selected". The honoured gate of `FrameNotOpened` is what makes that reading sound: only an opened frame sets `honoured`, so a non-empty slot cannot arise from a frame that did not render. Remove the gate and the implication fails — a not-opened frame sets `honoured` while `selected` still holds ImGui's tab-0 default (Section 11, defect (e)).

Invariant 4 — deadlock freedom. No reachable state leaves the machine unable to progress. The three opened-frame cases partition the opened case and `FrameNotOpened` is enabled unconditionally, so a render step — opened or not — is enabled in every state.

Echo suppression. A Hub-driven active-tab change fires nothing. This is not a separate reachability goal: `Declare`, `Repush`, and `RemoveReadd` never fire, and each resets `honoured` to empty, so the frame that follows is always `FrameForceSelect`, which cannot fire. A failure to suppress an echo is therefore a spurious fire, caught by invariant 1. The fidelity traces for defects (b) and (c) are exactly echo-suppression failures.

10 Verification

Type-checking is a `fuzz` pass:

```
fuzz -t docs/tab_bar_selection.tex
```

Invariants 1, 2, and 3 are state properties, checked by asking ProB whether their negation is *reachable*. A goal that is not found means the invariant holds on every reachable state:

```
# Invariant 1 (must report: goal NOT found)
probcli docs/tab_bar_selection.tex -model_check -nodead \
  -goal "spurious=ztrue" -p DEFAULT_SETSIZE 2

# Invariant 2 (must report: goal NOT found)
probcli docs/tab_bar_selection.tex -model_check -nodead \
  -goal "fires>1" -p DEFAULT_SETSIZE 2

# Invariant 3 (must report: goal NOT found)
```

```
probcli docs/tab_bar_selection.tex -model_check -nodead \
  -goal "honoured/={} & selected/=active & clicked=zfalse" \
  -p DEFAULT_SETSIZE 2
```

Invariant 4 is the deadlock check over the full reachable state space:

```
# Invariant 4 (must report: no deadlock, no invariant violation)
probcli docs/tab_bar_selection.tex -model_check -p DEFAULT_SETSIZE 2
```

The opened dimension adds the `FrameNotOpened` operation but no new goal: because its stale honour surfaces as a first-frame-honour failure that then fires spuriously, defect (e) is caught by the invariant 3 and invariant 1 goals already listed. All four checks return the same verdict under the extended machine as before it — no counter-example, and `FrameNotOpened` covered among the operations.

11 Fidelity: reproducing the five defects

A model that cannot reproduce the known defects proves nothing. Each defect is the current specification with one guard removed; each then makes the corresponding invariant's negation reachable. Below, each variant names the edit, the goal that must change from *not found* to *found*, and the shortest offending trace. Under `DEFAULT_SETSIZE 2`, `TABID = {TABID1, TABID2}` with `home = TABID1`; write $t_1 = \mathit{home}$ and t_2 for the other id.

Defect (a) — first-frame honoured defaulting to active. Edit: `Declare` seeds `honoured' = {v?}` instead of $\varnothing$, modelling the original code reading the honoured slot's default as `active` rather than `_UNHONOURED`. Goals found: both invariant 3 (`honoured/={} & selected/=active & clicked=zfalse`) and invariant 1 (`spurious=ztrue`). Trace:

1. `Init`: `active = selected =` t_1.
2. `Declare(`t_2`)` in the buggy form: `active =` t_2, `selected =` t_1, `honoured =` $\{t_2\}$, `clicked = zfalse`. This state alone violates invariant 3: `honoured` is non-empty, the user has not clicked, yet `selected =` $t_1 \neq t_2$ `= active` — ImGui's tab-0 default stands where the declared tab should be.
3. `FrameFire`: `active =` $t_2 \in$ `honoured`, `selected =` $t_1 \neq t_2$, $t_1 \notin$ `pending`, so the frame fires with `clicked = zfalse` — `spurious` is set, violating invariant 1.

Under the intact guard, `Declare(`t_2`)` leaves `honoured =` $\varnothing$, so the first frame is `FrameForceSelect`: it drives `selected` to t_2 and fires nothing.

Defect (b) — honoured surviving a re-push. Edit: `Repush` omits `honoured' =` $\varnothing$, keeping `honoured' = honoured` (the original `reset_honoured` not running on the re-push path). Goal found: invariant 1 (`spurious=ztrue`). Trace:

1. `Init`; `FrameForceSelect`: `honoured =` $\{t_1\}$, `selected = active =` t_1.
2. `UserClick(`t_2`)`: `selected =` t_2, `clicked = ztrue`.
3. `FrameFire`: fires once (legitimately), `pending =` $\{t_2\}$, `honoured =` $\{t_1\}$.
4. `Repush(`t_1`)` in the buggy form — a re-push that does not change this bar's active tab: `active =` t_1, `honoured` kept as $\{t_1\}$, `pending =` $\varnothing$, `clicked = zfalse`, `selected =` t_2 kept.
5. `FrameFire`: `active =` $t_1 \in$ `honoured`, `selected =` $t_2 \neq t_1$, $t_2 \notin$ `pending`, so it fires with `clicked = zfalse` — `spurious` is set. The stale honoured value made the re-push's own selection read as a fresh user switch.

Under the intact guard, `Repush` empties `honoured`, so the next frame is `FrameForceSelect`: it drives `selected` back to the Hub's t_1 and fires nothing.

Defect (c) — honoured not cleared on removal. Edit: `RemoveReadd` omits `honoured' =` $\varnothing$ (the original `discard_for` not clearing the honoured key). Goal found: invariant 1. The trace is that of defect (b) with `RemoveReadd(`t_1`)` in place of `Repush(`t_1`)` at step 4: a same-id bar re-added while its stale honoured value survives reads its stale ImGui selection as a fresh switch and fires spuriously on re-add. Under the intact guard, the re-add empties `honoured` and the next frame force-selects without firing.

Defect (d) — honoured not advanced on a user switch (the OPEN finding). Edit: `FrameFire` drops the `selected ∉ pending` guard and the `pending' = {selected}` write, modelling the original renderer that wrote only the active tab into the honoured slot and kept no record of the tab it had already fired for. Goal found: invariant 2 (`fires>1`). Trace:

1. `Init`; `FrameForceSelect`: `honoured` = $\{t_1\}$, `selected = active` = t_1.
2. `UserClick`(t_2): `selected` = t_2, `clicked = ztrue`, `fires = 0`.
3. `FrameFire`: `active` = $t_1 \in$ `honoured`, `selected` = $t_2 \neq t_1$, so it fires — `fires = 1`. In the buggy form nothing records t_2 as pending.
4. `FrameFire` again — the next frame in the same latency window, before any re-push, with `active` = t_1, `honoured` = $\{t_1\}$, `selected` = t_2 unchanged: the guard that would have seen t_2 pending is gone, so it fires *again* — `fires = 2`, violating invariant 2. Every further frame in the window re-fires.

Under the intact guard, step 3 records `pending` = $\{t_2\}$, so step 4 is `FrameSteady`: `selected` = $t_2 \in$ `pending`, no fire. The window fires exactly once.

Defect (e) — honoured recorded on a frame that did not render. Edit: `FrameNotOpened` writes `honoured' = {active}` instead of `honoured' = honoured`, modelling `end()` calling `record_honoured` unconditionally rather than under the `opened` guard — so a frame in which `begin_tab_bar` returned not opened, and no tab item ran, still records the active tab as honoured. Goals found: both invariant 3 (`honoured/={} & selected/=active & clicked=zfalse`) and invariant 1 (`spurious=ztrue`). Trace, for a bar declared with active tab t_2 inside a collapsed `collapsing_header`:

1. `Init`: `active = selected` = t_1.
2. `Declare`(t_2): `active` = t_2, `selected` = t_1 (ImGui's tab-0 default), `honoured` = $\varnothing$, `clicked = zfalse`. The bar is nested in the collapsed header, so no frame has yet opened it.
3. `FrameNotOpened` in the buggy form — a frame passes while the header is collapsed: `begin_tab_bar` returns not opened, no tab is force-selected, yet `end()` records `honoured` = $\{t_2\}$. This state alone violates invariant 3: `honoured` is non-empty, the user has not clicked, yet `selected` = $t_1 \neq t_2$ = `active` — the declared tab was never force-selected, because nothing rendered.
4. `FrameFire` — the header is expanded and the bar opens for the first time: `active` = $t_2 \in$ `honoured`, so the first-frame force-select is skipped; `selected` = $t_1 \neq t_2$, $t_1 \notin$ `pending`, so the frame fires with `clicked = zfalse` — `spurious` is set, violating invariant 1.

Under the intact gate, `FrameNotOpened` leaves `honoured` = $\varnothing$, so the first frame that opens the bar is `FrameForceSelect`: it drives `selected` to t_2 and fires nothing. ProB's own shortest witness for invariant 3 takes a different route to the same stale honour — `UserClick`(t_2); `Repush`(t_1); `FrameNotOpened`, where the re-push's own selection is left unreconciled by a frame that does not render — confirming the defect is a property of the unconditional honoured write, not of one particular path to it.

Summary. Each defect's guard, removed, makes its invariant's negation reachable; with every guard intact, the three reachability goals (`spurious=ztrue`, `fires>1`, and the invariant 3 predicate) all return *not found* and the deadlock check reports none. That difference is the evidence that the model detects the defects the code was changed to prevent — for defect (d), that the `pending` slot is the correct fix (the minimal state that makes the latency window fire exactly once), and for defect (e), that gating the honoured write on `opened` is the correct fix (only a frame that renders may record a honour, so `honoured` $\neq \varnothing$ remains a true witness that an opened frame force-selected).

12 From the Model to the Code

The model's `pending` slot is a second echo-suppression key in `WidgetState`, beside the honoured key. `begin_tab` reads it, fires only when the reported selection is neither the active tab nor the pending tab, and records the fired tab into it. The re-push and removal resets (`reset_honoured`, `discard_for`)

clear the pending key alongside the honoured key, which is the model's `pending' = ∅` in `Repush` and `RemoveReadd`.

The `FrameNotOpened` gate is `end()`'s honoured write moving inside its `if opened:` branch. `end()` already receives the `opened` flag that `begin` (`begin_tab_bar`) returned; the correction is that `record_honoured` runs only when that flag is set, so a frame in which the bar did not render — a collapsed `collapsing_header` or a hidden window — records no honour and the honoured slot keeps its meaning: the active tab that an *opened* frame last force-selected. This is the model's `honoured' = honoured` in `FrameNotOpened`, as against the opened cases' `honoured' = {active}`. No other honoured- or fire-related behaviour changes.